

Déploiement d'un modèle de langage (LLM) avec Docker et Kubernetes

Yacine Mbaye
École Polytechnique de Thiès
DIC2 – Génie Informatique et Télécommunications

Septembre 2026

1 Introduction

Ce projet consiste à déployer une pile applicative composée d'Ollama, moteur d'exécution de modèles de langage en local, et d'Open WebUI, interface web de discussion placée par-dessus. Le déploiement se fait en deux temps : d'abord avec Docker Compose, comme socle de développement rapide, puis par une migration vers Kubernetes respectant les bonnes pratiques d'orchestration (persistance des données, santé des services, configuration externalisée, exposition réseau). Trois modèles ont été rendus disponibles : Llama 3.2 (3B), Mistral (7B) et Gemma 3 (4B).

2 Socle Docker Compose

L'architecture de départ repose sur deux services définis dans `docker-compose.yaml`. Le service `ollama` expose l'API d'inférence sur le port 11434, avec un volume nommé qui évite de re-télécharger les modèles à chaque redémarrage. Le service `open-webui` sert l'interface web et pointe vers Ollama via la variable `OLLAMA_BASE_URL`. Les deux services communiquent sur un réseau bridge commun, avec un `healthcheck` Docker natif et une condition `depends_on: service_healthy` qui garantit qu'Open WebUI ne démarre qu'une fois Ollama réellement opérationnel.

3 Migration vers Kubernetes

3.1 Isolation dans un namespace

Toutes les ressources sont regroupées dans le namespace `llm-devops`, séparé du reste du cluster. Cette isolation évite les collisions de noms et permet de supprimer l'ensemble du déploiement d'un coup, sans risque pour le reste du cluster.

3.2 Configuration externalisée

Les variables d'environnement ne sont pas codées en dur dans les Deployments : elles sont portées par des ConfigMaps, une pour Ollama (`OLLAMA_HOST`, `OLLAMA_KEEP_ALIVE`) et une pour Open WebUI (`OLLAMA_BASE_URL`), injectées via `envFrom`. La configuration peut ainsi changer sans reconstruire une image ni modifier le Deployment.

3.3 Persistance des données

Un PersistentVolumeClaim (PVC) Kubernetes est un espace de stockage totalement indépendant du volume Docker utilisé par Compose ; aucune migration automatique n'existe entre les deux. Au premier déploiement Kubernetes, ni les modèles Ollama ni le compte Open WebUI n'étaient présents, ce qui a nécessité un nouveau téléchargement des modèles. Un PVC de 15 Gi a été alloué à Ollama et un de 5 Gi à Open WebUI.

3.4 Deployments et probes de santé

Chaque service dispose de son propre Deployment, avec des `requests/limits` de CPU et mémoire pour éviter qu'un service ne monopolise les ressources du nœud. Trois probes complètent le `healthcheck` unique de Compose : une `startupProbe` (jusqu'à 150 secondes pour démarrer, utile pour Ollama), une `readinessProbe` (retire le Pod du Service tant qu'il n'est pas prêt, sans le redémarrer) et une `livenessProbe` (redémarre le conteneur s'il cesse de répondre).

3.5 Exposition des services

Un Service est créé pour chaque Pod, fournissant un nom DNS stable (`ollama`, `open-webui`) qui ne change pas même si l'IP du Pod change. La variable `OLLAMA_BASE_URL` pointe vers `http://ollama:11434`, résolu automatiquement par le DNS interne du cluster (CoreDNS).

3.6 Récupération et test des modèles

Le PVC étant neuf au premier déploiement, les trois modèles ont été téléchargés directement dans le Pod, puis testés individuellement depuis l'interface Open WebUI pour confirmer qu'ils généraient bien une réponse.

3.7 Accès via un nom de domaine

Plutôt qu'un accès par port (NodePort), un Ingress expose Open WebUI sous le nom d'hôte `llm-devops.local`, via le contrôleur Nginx fourni par l'addon Minikube. Sur macOS avec le driver Docker, `minikube tunnel` doit rester actif en parallèle pour que ce nom d'hôte, déclaré dans `/etc/hosts`, pointe correctement vers le cluster.

4 Docker Compose face à Kubernetes

La migration met en évidence plusieurs différences structurelles entre les deux plateformes.

Aspect	Docker Compose	Kubernetes
Santé des services	<code>healthcheck</code> unique par conteneur	Trois probes distinctes (<code>startup</code> / <code>readiness</code> / <code>liveness</code>)
Réseau	Réseau bridge partagé, résolution par nom de service	Services + DNS interne (CoreDNS), indépendant de l'IP des Pods
Stockage	Volumes nommés, locaux au moteur Docker	PersistentVolumeClaim, provisionné indépendamment du cycle de vie du Pod
Auto-réparation	Redémarrage du conteneur seul	Recréation complète du Pod par le Deployment, avec vérification de santé
Mise à l'échelle	Manuelle (<code>docker compose scale</code>)	Déclarative (<code>replicas</code>), combinable avec un HPA

TABLE 1 – Comparaison structurelle entre Docker Compose et Kubernetes

Le stockage est le point le plus structurant : un volume Docker et un PVC Kubernetes sont deux systèmes complètement séparés, ce qui explique la perte de données observée lors de la première migration.

5 Travaux bonus réalisés

- **PodDisruptionBudget**

Un PDB protège les Pods contre les coupures volontaires (maintenance planifiée, `kubectl drain`). Avec une seule réplique par service, `minAvailable: 1` interdit toute éviction volontaire, ce qui illustre l'intérêt des PDB surtout en présence de plusieurs répliques.

- **Kustomize**

Les manifestes ont été réorganisés en une base commune (`k8s/base`) et deux overlays : `dev`, identique à la base, et `prod`, qui augmente les `requests/limits` CPU et mémoire via des patches.

- **Pipeline CI/CD**

Le pipeline GitHub Actions comprend deux jobs : `validate`, qui vérifie la syntaxe et le schéma des manifestes via `kubeconform` sur un runner GitHub classique, et `deploy`, qui applique réellement les changements sur le cluster Minikube local via un runner self-hosted. Ce second job est volontairement restreint aux push directs sur `main`, jamais aux Pull Requests, pour éviter qu'un tiers ne fasse exécuter du code sur la machine hôte via une PR.

- **Supervision Prometheus/Grafana**

Compte tenu des ressources limitées du nœud Minikube, la stack complète `kube-prometheus-stack` a été volontairement écartée au profit de l'essentiel : Prometheus interroge directement le `cAdvisor` de chaque nœud via le proxy de l'API Kubernetes, et Grafana est préconfiguré pour pointer vers Prometheus dès le démarrage.

- **Autoscaling horizontal (HPA)**

Un HPA a été configuré par Deployment, avec des seuils de 70 % de CPU et 80 % de mémoire, pour une plage d'une à trois répliques. Une montée en charge observée sur Open WebUI a effectivement déclenché un passage à trois répliques. Point à surveiller : les PVC étant en `ReadWriteOnce`, il conviendrait de vérifier qu'aucune réplique supplémentaire n'entre en conflit en écrivant sur les mêmes données.

- **NetworkPolicy**

Par défaut, tous les Pods d'un cluster Kubernetes peuvent se parler librement, ce qui n'est pas idéal du point de vue sécurité. Quatre politiques ont été mises en place : `default-deny-all`, qui bloque tout le trafic entrant par défaut, `allow-dns-egress`, qui préserve la résolution DNS, puis deux règles ciblées qui n'ouvrent que les flux strictement nécessaires vers Ollama et vers Open WebUI.

6 Conclusion

Ce projet a permis de mettre en œuvre une migration complète d'une architecture applicative de Docker Compose vers Kubernetes, en couvrant l'ensemble des primitives attendues (Namespace, ConfigMap, PVC, Deployment, Service, Ingress, probes) ainsi que six des sept bonus proposés, menés jusqu'à leur validation effective.